Dry run and formal run

Many regulated teams, in pharmacovigilance (PV) and in banking and insurance, test twice. A **dry run** checks the scripts against the final build, so no surprise turns up later. A **formal run** then executes the same scripts against the same build and records the evidence that is kept for audit. This guide explains how the framework supports that, how to switch it on, and how a team ends up with one zip holding everything.

None of this is active until regulated mode is switched on. Everyday runs, and the standard reports, do not change.

What it adds

Item	What it does
Run labels	A run is a **dry** or **formal** run of a named **build**. The label is in the manifest, in a banner at the top of the report, and on every screenshot and video frame.
Archive zip	The whole evidence pack is sealed into one zip with a SHA-256 checksum file, never overwritten, and every action is added to a tamper-evident audit log.
Comparison	A formal run is checked against the accepted dry run: same build, same scripts, same tests, same verdict for every requirement. A formal run that differs is not sealed.
CI controls	A manual workflow with a protected environment for formal runs, which can only start from a release tag.
Change record	What the code, the configuration and (optionally) the application looked like at the start and end of the run, plus notes written while it ran.

Switch it on

The settings are in `config/regulated.json`. It is committed, so a change to it is reviewed by the code owners and kept in git.

```
{
  "enabled": false,
  "features": { "runLabels": true, "archive": true, "compare": true, "ciControls": true },
  "formalRun": {
    "requireBuildId": true,
    "requireCleanTree": true,
    "requireDryRun": true,
    "allowLocal": true,
    "allowedRefs": ["refs/tags/*"]
  },
  "compare": {
    "dryMustPass": true,
    "requireSameBuild": true,
    "requireSameScripts": true,
    "requireSameTests": true,
    "requireSameVerdicts": true
  },
  "archive": { "directory": "reports/archive" },
  "changeRecord": { "probes": [] }
}
```

- Set `"enabled": true` to switch everything on. Any single feature can be switched off under `features`.
- `REGULATED_MODE=on` or `off` overrides the file for one run, or one CI environment. The order of precedence is the environment variable, then the file, then the built-in default (off).
- `npm run regulated:status` shows what is active and where that decision came from.

Setting	Meaning
`formalRun.requireBuildId`	every run needs `--build`
`formalRun.requireCleanTree`	a formal run refuses to start with uncommitted changes
`formalRun.requireDryRun`	a formal run must name an accepted dry run (`--against`)
`formalRun.allowLocal`	whether a formal run may be started from a local machine
`formalRun.allowedRefs`	where a formal run in CI may start from. The default is a release tag
`compare.dryMustPass`	the dry run must have no failed tests or requirements
`compare.requireSame...`	build, scripts (the code fingerprint), tests, requirement verdicts. A different commit with identical scripts, for example one that only changes documents, is a warning, not a failure
`changeRecord.probes`	URLs (a status or version page) fetched at the start and the end, to notice a change during the run

The process

```
npm run regulated:run -- --type dry --build 2.4.1
npm run regulated:run -- --type formal --build 2.4.1 --against reports/archive/DRY-2.4.1-<time>.zip
```

Each command does everything in order: checks the rules, clears old results, runs the whole suite in capture mode (a screenshot, video and trace of every test, each page stamped with the run label and a live clock), builds the evidence pack, compares a formal run with its dry run, and seals the result.

1. **Dry run.** Run it against the final build. Read `validation-report.html` in the pack. If there are problems, fix them and run again. A dry run is labelled "not audit evidence" everywhere.
2. **Accept the dry run.** The team decides the dry run is satisfactory. Its zip is the reference.
3. **Formal run.** Run it against the same build, from committed code, with `--against` pointing at the accepted dry run zip. It is refused straight away if the working tree has uncommitted changes, if the dry run is for a different build, or if the dry run's zip does not pass its own integrity check.
4. **Comparison.** The formal run is compared with the dry run. If anything differs, the formal pack is built for investigation but **not sealed**.
5. **Seal.** A sealed zip and its checksum file appear in `reports/archive`. Keep them together in your records.

To run only some tests, add the usual Playwright options, for example `--grep @smoke`. The formal run must run the same tests as its dry run, or the comparison will say so.

Dry run on a work in progress

A dry run may be started with uncommitted changes. It records them and prints a warning, because such a dry run cannot be the basis of a formal run until the code is committed and unchanged.

When a run fails

A run with failing tests is still sealed, with `-FAILED` at the end of its name, so a record of the attempt exists and nobody mistakes it for a good run. A formal run that is not accepted, because it differs from the dry run, is not sealed at all.

Try it first

You can try the whole process without changing any file, on a few tests, in a few minutes:

```
REGULATED_MODE=on npm run regulated:run -- --type dry --build 1.0 --grep "a customer can sign in|valid credentials return"
REGULATED_MODE=on npm run regulated:run -- --type formal --build 1.0 --against reports/archive/DRY-1.0-<time>.zip --grep "a customer can sign in|valid credentials return"
```

(On Windows PowerShell, set the variable first: `$env:REGULATED_MODE='on'`.)

- Use the **same** `--grep` for both runs. A formal run that ran different tests is not accepted, which is the point.
- The formal run needs a clean working tree, so commit or discard your changes first.
- With only a few tests, many requirements show as "not tested". That is expected for a trial.

Reviewing a run

After a run, the results are in two places:

Where	What
`reports/archive/`	The sealed zips (`DRY-...zip` , `FORMAL-...zip`), each with a `.sha256` file, and `audit-log.jsonl` . This is what you keep.
`reports/regulated/`	The latest run, unzipped, so you can look at it straight away. The next run replaces it.

To review a run:

1. Open `validation-report.html` (in `reports/regulated` , or unzip a sealed archive and open it there). The banner at the top says dry or formal, the build and the execution key.
2. Section 8, "Changes and notes outside the run", shows the git state and code fingerprint at the start and the end, and any notes.
3. Section 9, "Comparison with the dry run" (formal runs), shows each check.
4. For the steps, screenshots, videos and traces of a test, open the Allure report: `npm run report:validation:open` .
5. Check the archive itself with `npm run regulated:verify -- <zip>` .

Cleaning up

`npm run clean` removes `reports/_raw` , `reports/standard` and `reports/regulated` . It **leaves** `reports/archive` **alone**, because sealed archives are audit records. Copy the archives you need to your records first. To remove them on purpose, delete the folder or run `node tools/clean-reports.mjs archive` .

What ends up in the zip

The sealed zip holds the whole evidence pack. Open `validation-report.html` first.

In the zip	What it is
`validation-report.html`	one page: overall result, run type and build banner, traceability, known defects, all results, changes and notes, comparison with the dry run
`validation-report.pdf`	the same page as a PDF
`allure-results/`	raw results, with every screenshot, video and trace
`allure/`	the Allure report of the run
`traceability/`	the requirement-to-test matrix (HTML, CSV, Markdown)
`change-record.json`	the change record (see below)
`comparison/`	the comparison with the dry run (formal runs)
`manifest.json`	framework version, commit, build, run type, execution key, who ran it, tool versions, image digests, the settings in force
`SHA256SUMS`	a checksum for every file in the zip

Next to the zip is `<name>.zip.sha256`, the checksum of the zip itself. Store both in Xray and SharePoint. The electronic signature is applied in your quality system; quote the zip's SHA-256 in the signature record.

Checking an archive later

```
npm run regulated:verify -- reports/archive/FORMAL-2.4.1-<time>.zip
npm run regulated:verify -- reports/archive/audit-log.jsonl
```

The first command checks the zip against its `.sha256` file and every file inside against the zip's own `SHA256SUMS`. It names anything missing, changed or added. The second checks the audit log. This needs no setup beyond Node, and does not need regulated mode, so an auditor or a colleague can run it on a copy of the zip.

Changes made outside the run

A local run is only as trustworthy as the machine and the person. The change record narrows the gap.

Recorded, at the start and the end of the run
The commit, branch and tag, whether the working tree is clean, the list of changed files, and how the commit compares with the remote branch
A **code fingerprint**: a SHA-256 over the tests, source, configuration, tools and requirement list, with a hash per area, so a difference names the area
A hash of the environment profile file (the hash only, never the contents) and the non-secret settings: URLs, browsers, workers
Optional **application probes**: URLs listed under `changeRecord.probes`, fetched at the start and the end, with the status and a hash of the answer

The report lists every difference between the start and the end, such as a commit that moved, a file that changed or a probe that answered differently.

Notes. For anything done by hand while a run is in progress, such as restarting the application, re-seeding data or waiting for an outage, add a note:

```
npm run regulated:note -- "restarted the application container at 10:12 after it stopped answering"
```

The note is stored with the time and the person, and appears in the report.

The audit log. Every regulated run, sealing and refusal is appended to `reports/archive/audit-log.jsonl`. Each line holds the hash of the line before it, so an edited or deleted line breaks the chain, and `regulated:verify` shows it.

What this cannot show. Edits made to systems outside the repository, such as a direct change to a database, are invisible unless a probe or a note records them. On a local machine the log and the notes are only as honest as the person using them. A CI formal run, which starts from a tag and needs an approval, is stronger evidence.

In CI

A manual workflow, **QA Accelerator Regulated Run**, does the same thing on GitHub. Set it up once in the repository settings:

1. Create two environments (Settings, Environments): `dry-validation` with no rules, and `formal-validation` with **Required reviewers**, so a named person must approve every formal run.
2. Add the four credential secrets, as for the normal workflow.
3. Set `"enabled": true` in `config/regulated.json`.

Then run it from the Actions tab:

Input	For
`run_type`	`dry` or `formal`
`build_id`	the application build or version (letters, digits, dots, dashes, underscores)
`dry_run_id`	formal only: the run ID of the accepted dry run, the number in its address
`execution_id`	optional: an Xray Test Execution key, recorded in the evidence

A **dry** run can start from any branch. A **formal** run must start from a tag (choose the release tag under "Use workflow from"), waits for the reviewers, downloads the dry run's archive, runs, compares and seals. The result is uploaded as the artifact `regulated-<type>-<build>`, holding the zip, its checksum and the audit log. GitHub keeps artifacts for a limited time, so copy an approved formal archive into Xray and SharePoint and check it with `npm run regulated:verify`.

If regulated mode is off, the workflow stops at its first job with a message saying so.

Limits

- The framework supports your process; it does not replace your quality system. Signing, storage and retention stay there.
- Dry and formal runs are comparable only if they used the same machine type, the same tool versions, the same framework version and the same application images. The comparison reports a difference in any of these as a warning.
- A local formal run depends on the person running it. The clean-tree rule, the code fingerprint, the notes and the audit log reduce that risk; they cannot remove it.